

Agentes

Evolución, arquitectura, evaluación, protocolos y futuro

Jorge Ortiz Fuentes

23 de junio de 2025

Contexto histórico

Introducción y evolución de los agentes

Arquitectura y mecanismos de razonamiento

Interoperabilidad y colaboración

Aplicaciones y casos de estudio

Ventajas y desafíos

Evaluación de sistemas de agentes

Contexto histórico

Las raíces del concepto de agente

Orígenes en la IA simbólica

El concepto de agente no es nuevo. Sus fundamentos teóricos se remontan a la investigación en **Inteligencia Artificial Distribuida** de los años 70 y 80, mucho antes de la era de los LLM (Krishnan, 2025).

De la investigación al consumidor

En los años 90, la tecnología de agentes comenzó a salir de los laboratorios para integrarse en el software comercial.

- **1990:** Se lanza “Dragon Dictate”, el primer software de reconocimiento de voz para consumidores, marcando un hito en la interacción humano-computadora (Y. Li et al., 2024).
- **1993:** Apple introduce “Speakable Items”, que permite a los usuarios controlar su ordenador mediante comandos de voz en lenguaje natural, integrando la agencia a nivel de sistema operativo (Y. Li et al., 2024).

La era de Internet: La integración con aplicaciones

Hacia agentes más funcionales

A finales de los 90 y en los 2000, los agentes comenzaron a especializarse y a conectarse con servicios externos.

- **1996:** IBM lanza “MedSpeak”, un software de reconocimiento de voz continuo diseñado específicamente para radiólogos, demostrando el potencial de los agentes en dominios especializados (Y. Li et al., 2024).
- **2008:** Google añade la búsqueda por voz a su aplicación móvil, sentando las bases para los asistentes modernos al conectar el reconocimiento de voz con un servicio de información masivo (Y. Li et al., 2024).

El concepto de Asistente Personal Inteligente (IPA)

Definición de IPA

Los Asistentes Personales Inteligentes (IPAs) son agentes de software diseñados para ayudar a individuos a realizar tareas y obtener información de manera eficiente.

Evolución a la era móvil

La proliferación de dispositivos personales (smartphones, etc.) expandió las fronteras funcionales de los IPAs y se transformaron de simples comandos de voz a servicios siempre conectados e integrados con nuestros datos personales (Y. Li et al., 2024).

Los IPAs basados en plantillas

- Asistentes como Siri (2011), Google Assistant y Alexa representan la primera generación de agentes comerciales masivos.
- Funcionaban principalmente con un enfoque basado en **plantillas y reglas predefinidas**.
- Los desarrolladores debían especificar explícitamente las funciones, los disparadores y los pasos de ejecución para cada tarea (Y. Li et al., 2024).

El problema de la escalabilidad y la flexibilidad

Los sistemas basados en plantillas, aunque fiables, sufrían de problemas críticos:

- **Escalabilidad limitada:** Soportar nuevas tareas requería un costo enorme de tiempo y trabajo manual, lo que restringía su utilidad.
- **Falta de flexibilidad:** Los agentes no podían comprender ni ejecutar tareas que estuvieran fuera de sus plantillas predefinidas.
- **Experiencia de usuario frágil:** Los usuarios abandonaban fácilmente el uso de los IPAs tras varios intentos fallidos (Y. Li et al., 2024).

La llegada de los LLM: Un cambio de paradigma

La solución a los problemas de escalabilidad

La aparición de los modelos de lenguaje grandes (LLMs) ha brindado nuevas oportunidades para superar los desafíos históricos:

- Gracias a sus capacidades de **razonamiento de sentido común** y **generalización zero-shot**, los LLM pueden planificar y ejecutar tareas complejas sin necesidad de plantillas predefinidas o entrenamiento específico para cada tarea.
- Esto marca la transición de sistemas rígidos y limitados a los **agentes autónomos y flexibles** que estudiamos hoy.

Introducción y evolución de los agentes

Perspectiva 1: la definición clásica

Definición fundamental

Un agente es una entidad computacional que percibe su entorno a través de sensores y actúa sobre él mediante actuadores (citado en Krishnan, 2025; Russell & Norvig, 1995). A diferencia de un LLM, que es principalmente reactivo, un agente posee propiedades clave que le otorgan un comportamiento autónomo y dirigido.

Propiedades clave de la agencia

- **Autonomía:** opera sin intervención humana directa.
- **Reactividad:** responde a cambios en el entorno.
- **Proactividad:** exhibe comportamiento dirigido por objetivos.
- **Habilidad social:** interactúa con otros agentes y humanos.

(citado en Krishnan, 2025; Wooldridge & Jennings, 1995)

Perspectiva 2: la definición funcional

Un agente razona, actúa e interactúa

Define a los agentes según las tres funciones principales que realizan para lograr sus objetivos (Plaet et al., 2025).

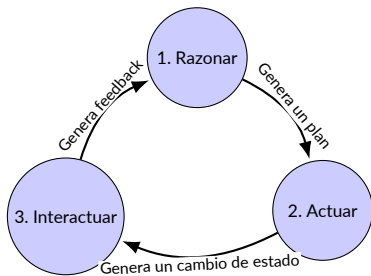


Figura 1: El ciclo funcional de un agente de IA.

En resumen

No existe una única definición de “agente”. El énfasis depende de sus:

- **Propiedades** (autonomía, reactividad)
- **Funciones** (razonar, actuar, interactuar)
- **Comportamientos** (simulados)

Diferencias clave: Agente vs. Asistente vs. Bot

	Agente de IA	Asistente de IA	Bot
Objetivo	Realizar tareas de forma autónoma y proactiva	Ayuda a los usuarios con las tareas	Automatización de tareas o conversaciones simples
Funciones	Puede realizar acciones complejas de varios pasos, aprende, se adapta y puede tomar decisiones de forma independiente	Responde a solicitudes o instrucciones, proporciona información y completa tareas simples	Sigue reglas predefinidas; aprendizaje limitado; interacciones básicas
Interacción	Proactivo y orientado a los objetivos	Reactiva; responde a las solicitudes de los usuarios	Reactivo; responde a activadores o comandos

Adaptado de Google Cloud Google Cloud (2025).

Arquitectura y mecanismos de razonamiento

Componentes centrales del agente

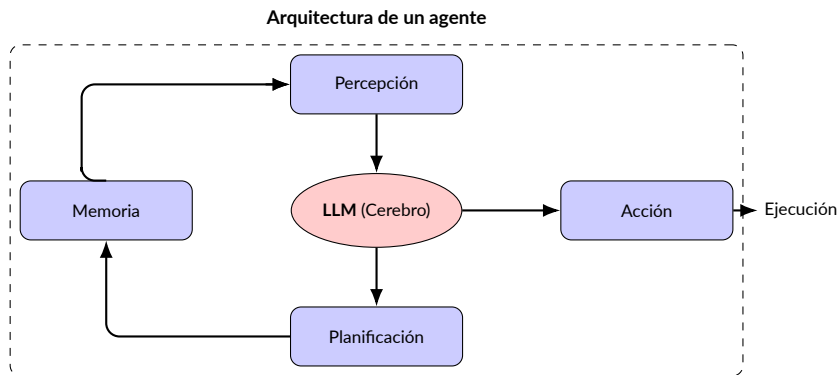


Figura 2: Arquitectura modular de un agente de IA, donde el LLM se coordina con los componentes de percepción, planificación, memoria, y acción (Huang et al., 2024; Park et al., 2023; Plaat et al., 2025; Xie et al., 2024).

Razonamiento por Cadena de Pensamiento (CoT)

Definición

Para resolver problemas complejos, los humanos suelen descomponerlos en pasos intermedios. *Chain-of-Thought* (CoT) es una técnica de *prompting* que induce a los LLM a emular este proceso, generando una secuencia de razonamiento explícita paso a paso (Wei et al., 2022).

Impacto

Un simple *prompt* como “Let’s think step-by-step” puede mejorar significativamente el rendimiento en tareas aritméticas, de sentido común y de razonamiento simbólico, ya que fuerza al modelo a no tomar atajos y a construir una respuesta lógica (citado en Plaat et al., 2025).

Razonamiento por autorreflexión (Self-reflection)

Definición

Los agentes pueden mejorar sus resultados al reflexionar sobre sus fallos y recibir retroalimentación. La autorreflexión es el mecanismo que permite a un agente evaluar su propio rendimiento y corregir errores de forma iterativa.

Framework *Reflexion*

Este proceso se puede dividir entre múltiples instancias de un LLM (Shinn et al., 2024):

1. Un **Actor** genera el texto y las acciones iniciales.
2. Un **Evaluador** califica los resultados del actor, identificando fallos.
3. Un **Autorreflector** genera retroalimentación verbal para guiar al actor en la siguiente iteración.

Memoria: la base de la coherencia

- El *memory stream* es un registro completo de las experiencias del agente, almacenado en lenguaje natural. Este es el componente que le otorga persistencia y contexto a largo plazo.
- La recuperación de recuerdos se basa en una combinación de:
 - **Recencia:** ¿Cuán reciente es el recuerdo?
 - **Importancia:** ¿Qué tan significativo fue el evento?
 - **Relevancia:** ¿Cómo se relaciona con la situación actual?
- Este mecanismo imita la memoria asociativa humana y permite al agente mantener una identidad y un comportamiento coherentes (Park et al., 2023).

Interoperabilidad y colaboración

La necesidad de un estándar

- Así como TCP/IP fue clave para crear una internet abierta y conectada, los agentes necesitan **protocolos estandarizados** para comunicarse entre sí y con herramientas externas (Yang et al., 2025).
- Sin un estándar, se crean “silos de inteligencia”, donde agentes de diferentes ecosistemas (OpenAI, Google, Anthropic, etc.) no pueden colaborar eficazmente, limitando su potencial (Ehtesham et al., 2025).

Model Context Protocol (MCP)

- **Propósito:** Estandarizar la comunicación entre **agentes y herramientas externas** (APIs, bases de datos).
- **Funcionamiento:** Proporciona un framework estructurado de entrada/salida para que los agentes soliciten y reciban contexto de manera segura (Jeong, 2025).
- **Característica clave:** Utiliza JSON Schema para definir claramente los tipos de entrada y salida de cada herramienta, garantizando la seguridad de los tipos y reduciendo errores de integración (citado en Jeong, 2025).

Propuesto por Anthropic (Anthropic, 2024)

¿Cómo funciona MCP?

- **Descubrimiento de herramientas:** El protocolo permite a un agente descubrir dinámicamente qué herramientas están disponibles y cómo usarlas a través de descripciones basadas en JSON Schema.
- **Llamada a funciones:** Estandariza el mecanismo para que un agente invoque funciones de herramientas externas y reciba sus resultados de manera estructurada.
- **Gestión de contexto:** Mantiene la información de estado durante el uso de la herramienta, lo que permite interacciones complejas de varios pasos (Jeong, 2025).

Agent-to-Agent Protocol (A2A)

- **Propósito:** Estandarizar la comunicación y colaboración entre múltiples agentes autónomos.
- **Funcionamiento:** Permite a los agentes descubrir las capacidades de otros, delegar tareas y gestionar el estado de dichas tareas.

Propuesto por Google y ahora bajo el gobierno de la Linux Foundation
(Google, 2025; The Linux Foundation, 2025)

¿Cómo funciona A2A?

- **Agent Card:** Una “tarjeta de presentación” en formato JSON que un agente publica para describir sus funciones, roles y capacidades. Funciona como un registro de servicios para el descubrimiento dinámico.
- **Task Management:** Un modelo de comunicación basado en tareas, que permite rastrear el estado ('created', 'in-progress', 'completed'), el progreso y los resultados (artefactos) de una solicitud.
- **Colaboración segura:** Incluye mecanismos de autenticación y autorización de nivel empresarial para el intercambio seguro de contexto y directivas (Jeong, 2025).

A2A: de iniciativa corporativa a estándar abierto

A2A se une a la Linux Foundation

En junio de 2025, se anunció que el protocolo A2A pasaría a ser un proyecto oficial de la **Linux Foundation**, el hogar de proyectos de código abierto críticos como Linux y Kubernetes (The Linux Foundation, 2025).

¿Por qué es importante este cambio?

- **Neutralidad:** al salir del control de una sola empresa, se fomenta una adopción más amplia y se evita el riesgo de *vendor lock-in*.
- **Gobernanza abierta:** asegura que el desarrollo del protocolo sea colaborativo, transparente y guiado por las necesidades de la comunidad.

Consolidación de un estándar industrial

Un ecosistema unificado

La transición a la Linux Foundation formaliza el A2A como un esfuerzo para crear un estándar industrial abierto, con el respaldo de actores clave del ecosistema tecnológico.

Respaldo de la industria

El proyecto cuenta con el apoyo y las contribuciones de líderes tecnológicos, tales como:

- Google Cloud, AWS, Microsoft Azure
- Salesforce, SAP, ServiceNow
- Cisco (a través de Outshift)

El respaldo masivo permite lograr una verdadera interoperabilidad (The Linux Foundation, 2025).

Complemento entre MCP y A2A

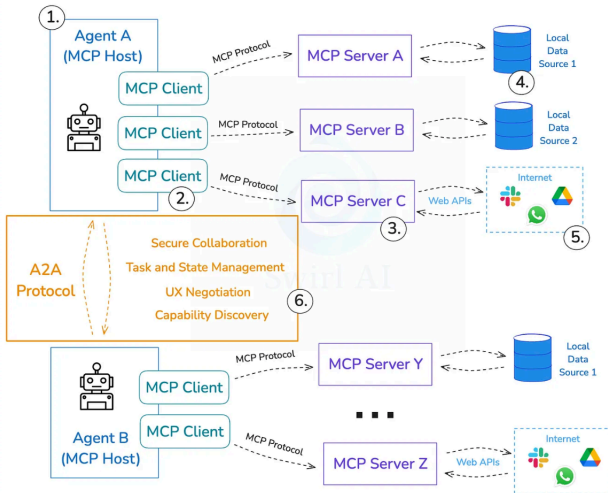


Figura 3: Extraído de Swirl AI Newsletter (Griciūnas, 2025).

Aplicaciones y casos de estudio

Customer Service and Support

Los agentes pueden gestionar consultas, resolver problemas comunes y escalar casos complejos, actuando sobre la base de enormes cantidades de información de productos (Krishnan, 2025).

Business Process Automation

A diferencia de la automatización tradicional, los agentes pueden adaptarse a variaciones, gestionar excepciones y optimizar flujos de trabajo sobre la marcha (citado en Jeong, 2025).

Ingeniería de Software

Agentes como SWE-Agent pueden resolver de forma autónoma problemas reales de repositorios de GitHub, interactuando directamente con el sistema de archivos para editar y ejecutar código (Jimenez et al., 2023).

Capacidades clave

- Comprensión de bases de código existentes.
- Planificación y ejecución de cambios en múltiples archivos.
- Validación de las soluciones mediante la ejecución de pruebas.

Automatización del proceso de reclutamiento

Un sistema multiagente puede automatizar todo el flujo de trabajo, donde cada agente se especializa en una tarea y colabora con los demás:

- Un **agente de análisis** revisa currículums (usando MCP para conectarse a una base de datos).
- Un **agente coordinador** gestiona entrevistas (usando MCP para interactuar con APIs de calendario).
- Un **agente de preparación** genera preguntas personalizadas (recibiendo contexto del primer agente vía A2A).

(Jeong, 2025)

Caso de estudio: Agentes Generativos en “Smallville”

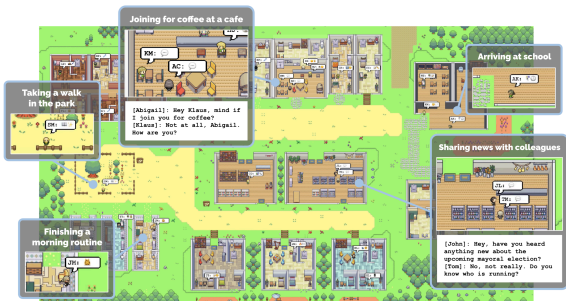


Figura 4: Una simulación con 25 agentes, cada uno con su propia personalidad y memoria, para estudiar la emergencia de dinámicas sociales complejas que no fueron explícitamente programadas (Park et al., 2023).

Comportamiento social emergente en “Smallville”

Comportamientos emergentes (no programados)

- **Difusión de información:** La noticia de una candidatura a la alcaldía se propagó orgánicamente.
- **Formación de relaciones:** Los agentes recordaron interacciones pasadas y formaron nuevas amistades.
- **Coordinación de actividades:** Un agente organizó una fiesta, y los invitados recordaron y asistieron de forma coordinada.

(Park et al., 2023)

Ventajas y desafíos

Ventajas clave de los agentes de IA

Autonomía y aprendizaje

- Automatizan tareas complejas de varios pasos sin intervención humana constante (Ferrag et al., 2025).
- Aprenden de la experiencia a través de la memoria y la autorreflexión, adaptándose a entornos dinámicos (Krishnan, 2025; Plaat et al., 2025).

Escalabilidad y colaboración

Los sistemas multiagente permiten descomponer problemas grandes y aprovechar la inteligencia colectiva, logrando resultados que un solo agente no podría alcanzar (X. Li et al., 2024).

Dependencia de la calidad del LLM

- Los agentes heredan las limitaciones de los LLM, como las alucinaciones y los sesgos (Krishnan, 2025).
- Un razonamiento erróneo en un paso puede afectar toda la cadena de planificación (Huang et al., 2024).

Seguridad y control

- Riesgo de *prompt injection* y manipulación por actores maliciosos.
- Dificultad para garantizar que las acciones del agente sean siempre seguras y alineadas con la intención del usuario (Y. Li et al., 2024).

Costo y latencia

- Las arquitecturas complejas, especialmente las multiagente, pueden tener un alto costo computacional debido a que son intensivas en tokens (X. Li et al., 2024).
- El proceso de percepción, razonamiento y acción introduce una latencia que puede no ser aceptable para todas las aplicaciones.

Evaluación de sistemas de agentes

¿Cómo sabemos si un agente es “bueno”?

Las métricas tradicionales como la tasa de éxito de la tarea son insuficientes y a menudo engañosas. Omiten aspectos críticos como:

- **Costo y eficiencia:** ¿cuántos tokens o llamadas a API consumió el agente?
- **Robustez:** ¿el agente falla ante entradas inesperadas o cambios en el entorno?
- **Seguridad y alineación:** ¿el agente cumple las restricciones y evita comportamientos dañinos?

(Kapoor et al., 2024; Yehudai et al., 2025)

Benchmarks: Hacia una evaluación más realista

La tendencia actual es crear entornos de prueba que simulen la complejidad y el dinamismo del mundo real, midiendo capacidades específicas más allá de la simple precisión.

Ejemplo de Benchmark: SWE-bench

Propósito

Evaluar agentes en la resolución de problemas de ingeniería de software **reales**, extraídos directamente de incidencias de repositorios de GitHub.

- Mide la capacidad del agente para comprender bases de código complejas, planificar cambios y ejecutar ediciones en múltiples archivos.
- Es una prueba de fuego para la aplicabilidad de los agentes en el desarrollo de software práctico.

(Jimenez et al., 2023; Yehudai et al., 2025)

Ejemplo de Benchmark: WebArena

Propósito

Mide la capacidad de los agentes para navegar e interactuar con **sitios web realistas y dinámicos** para completar tareas complejas.

- Las tareas incluyen reservar un vuelo, comprar productos o gestionar un calendario en sitios web funcionales.
- Evalúa la comprensión visual (con VisualWebArena) y la capacidad de interactuar con elementos de una GUI.

(Yehudai et al., 2025; Zhou et al., 2023)

Propósito

Presenta un conjunto de preguntas de razonamiento general que son **fáciles para los humanos pero extremadamente difíciles para la IA.**

- Requiere capacidades fundamentales como el razonamiento en varios pasos, el manejo de información multimodal y el uso de herramientas (navegación web, manipulación de archivos).
- Diseñado para medir el progreso hacia una inteligencia artificial más general y robusta.

(Mialon et al., 2023); citado en Yehudai et al. (2025)

Agentes que evalúan agentes

El uso de LLMs como “jueces” para evaluar las trayectorias y decisiones de otros agentes está ganando terreno como una solución escalable y rentable para la evaluación (Zhuge et al., 2024).

Direcciones futuras e investigación abierta

El futuro: hacia una inteligencia colectiva

El “Internet de los agentes”

Una red global y descentralizada donde miles de millones de agentes interactúan, colaboran y negocian para resolver problemas a una escala sin precedentes (Yang et al., 2025).

La IA como colaborador ubicuo

La visión a largo plazo es la de agentes que funcionan como socios personalizados en el aprendizaje, la creatividad y la resolución de problemas a lo largo de toda la vida humana (Krishnan, 2025).

Referencias

-  Anthropic. (2024). **Introducing the Model Context Protocol.**
<https://www.anthropic.com/news/model-context-protocol>
-  Ehtesham, A., Singh, A., Gupta, G. K., & Kumar, S. (2025). **A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP).**
-  Ferrag, M. A., Tihanyi, N., & Debbah, M. (2025). **From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review.**

-  Google. (2025). **Announcing the Agent2Agent Protocol (A2A)**.
<https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
-  Google Cloud. (2025). **What are AI agents?** Consultado el 23 de junio de 2025, desde
<https://cloud.google.com/discover/what-are-ai-agents>
-  Griciūnas, A. (2025). **MCP vs. A2A: Friends or Foes? [En: Swirl AI Newsletter]**. Consultado el 23 de junio de 2025, desde <https://www.newsletter.swirlai.com/p/mcp-vs-a2a-friends-or-foes>
-  Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., & Chen, E. (2024). **Understanding the planning of LLM agents: A survey.**

-  Jeong, C. (2025). **A Study on the MCP x A2A Framework for Enhancing Interoperability of LLM-based Autonomous Agents.**
-  Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2023). **SWE-bench: Can Language Models Resolve Real-World GitHub Issues?**
-  Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2024). **AI Agents That Matter.**
-  Krishnan, N. (2025). **AI Agents: Evolution, Architecture, and Real-World Applications.**
-  Li, X., Wang, S., Zeng, S., Wu, Y., & Yang, Y. (2024). **A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges.** *Vicinagearth*, 1(1), 9.

-  Li, Y., Wen, H., Wang, W., Li, X., Yuan, Y., Liu, G., Liu, J., Xu, W., Wang, X., Sun, Y., Kong, R., Wang, Y., Geng, H., Luan, J., Jin, X., Ye, Z., Xiong, G., Zhang, F., Li, X., ... Liu, Y. (2024). **Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security.**
-  Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., & Scialom, T. (2023). **GAIA: a benchmark for General AI Assistants.**
-  Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). **Generative Agents: Interactive Simulacra of Human Behavior.**
-  Plaat, A., van Duijn, M., van Stein, N., Preuss, M., van der Putten, P., & Batenburg, K. J. (2025). **Agentic Large Language Models, a survey.**

-  Russell, S. J., & Norvig, P. (1995). ***Artificial intelligence: a modern approach***. Prentice-Hall, Egnlewood Cliffs.
-  Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2024). **Reflexion: Language agents with verbal reinforcement learning.**
-  The Linux Foundation. (2025, junio). ***Linux Foundation Launches the Agent2Agent Protocol Project to Enable Secure, Intelligent Communication Between AI Agents***. Consultado el 23 de junio de 2025, desde <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>

-  Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). **Chain-of-thought prompting elicits reasoning in large language models.**
-  Wooldridge, M., & Jennings, N. R. (1995). **Intelligent agents: theory and practice.**
-  Xie, J., Chen, Z., Zhang, R., Wan, X., & Li, G. (2024). **Large Multimodal Agents: A Survey.**
-  Yang, Y., Chai, H., Song, Y., Qi, S., Wen, M., Li, N., Liao, J., Hu, H., Lin, J., Chang, G., Liu, W., Wen, Y., Yu, Y., & Zhang, W. (2025). **A Survey of AI Agent Protocols.**
-  Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., & Shmueli-Scheuer, M. (2025). **Survey on Evaluation of LLM-based Agents.**

-  Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2023). **WebArena: A Realistic Web Environment for Building Autonomous Agents.**
-  Zhuge, M., Zhao, C., Ashley, D., Wang, W., Khizbullin, D., Xiong, Y., Liu, Z., Chang, E., Krishnamoorthi, R., Tian, Y., Li, B., & Wang, C. (2024). **Agent-as-a-Judge: Evaluate Agents with Agents.**